

DATABASE NORMALIZATION

Pizza Sales Database – DBMS Project

1. Introduction

Database normalization is a systematic, technique-driven approach used in Relational Database Management Systems (RDBMS) to structure tables and fields. The primary objective of normalization is to minimize data redundancy and eliminate anomalies during data modification operations (insertion, update, and deletion anomalies). Normalization decomposes large, unorganized tables into smaller, cohesive, and logically connected relations.

In our Pizza Sales Database project, normalization ensures that core business entities—such as pizza categories, specific menu items (size and pricing), customer orders, and individual line items—are distinctly maintained. By preventing data duplication across transactional logs and menu definitions, the schema maintains strict referential integrity, improves query performance, and simplifies future database expansion.

2. Existing Database Schema

The database structure was retrieved directly from the project SQL schema and data source files. The schema consists of four relational tables linked through primary and foreign keys.

Table Name	Purpose	Attributes	Primary Key	Foreign Key(s)
pizza_types	Stores category metadata, full names, and ingredient descriptions.	pizza_type_id, name, category, ingredients	pizza_type_id	None
pizzas	Stores specific pizza variations with size options and pricing.	pizza_id, pizza_type_id, size, price	pizza_id	pizza_type_id → pizza_types.pizza_type_id
orders	Tracks high-level order header info (date & time).	order_id, date, time	order_id	None
order_details	Captures line-item quantities for ordered pizzas.	order_details_id, order_id, pizza_id, quantity	order_details_id	order_id → orders.order_id pizza_id → pizzas.pizza_id

3. Normalization Analysis

3.1 First Normal Form (1NF)

A relation is in First Normal Form (1NF) if every attribute contains only atomic (indivisible) values, and there are no repeating groups or multi-valued arrays within any row.

Analyzing the actual tables in our project:

- **order_details & orders:** Each record contains scalar integer values for order_id and order_details_id, scalar text strings for pizza_id, and single numeric values for quantity.
- **pizzas:** Attributes size and price contain indivisible values per record.
- **pizza_types:** The ingredients column contains a comma-separated list of ingredients. However, in our relational database model, this field is stored and queried as an atomic TEXT data type representing a static description, rather than a dynamic repeating array requiring relational join operations.

Conclusion: All attributes contain atomic values relative to the application domain, and each row is uniquely identifiable. Hence, the database satisfies First Normal Form (1NF).

3.2 Second Normal Form (2NF)

A relation is in Second Normal Form (2NF) if it is in 1NF and contains no partial dependencies (i.e., no non-key attribute is dependent on a proper subset of a composite primary key).

Analyzing the primary keys across our project tables:

- **pizza_types:** Primary key is pizza_type_id (Single attribute).
- **pizzas:** Primary key is pizza_id (Single attribute).
- **orders:** Primary key is order_id (Single attribute).
- **order_details:** Primary key is order_details_id (Single attribute).

Conclusion: Because every single table in the schema employs a single-attribute primary key, a partial dependency is mathematically impossible. Partial functional dependency can only occur when a primary key consists of two or more attributes (composite primary key). Since no composite primary keys exist in our database, the schema satisfies Second Normal Form (2NF).

3.3 Third Normal Form (3NF)

A relation is in Third Normal Form (3NF) if it is in 2NF and contains no transitive dependencies (i.e., no non-key attribute is functionally dependent on another non-key attribute). Every non-key attribute must depend directly on the primary key.

Analyzing functional dependencies across the schema:

- **pizza_types:** Attributes name, category, and ingredients depend solely on pizza_type_id.
- **pizzas:** Attributes size, price, and pizza_type_id depend directly on pizza_id. General pizza information (e.g., category, ingredients) is properly separated into pizza_types rather than being repeated inside the pizzas table.
- **orders:** Attributes date and time depend exclusively on order_id.
- **order_details:** Attributes order_id, pizza_id, and quantity depend directly on order_details_id. Neither order timestamp nor pizza price is stored here; they are referenced strictly via foreign keys.

Conclusion: No non-key attribute determines any other non-key attribute in any table. Every non-key field depends strictly on the primary key, whole primary key, and nothing but the primary key. Thus, the database satisfies Third Normal Form (3NF).

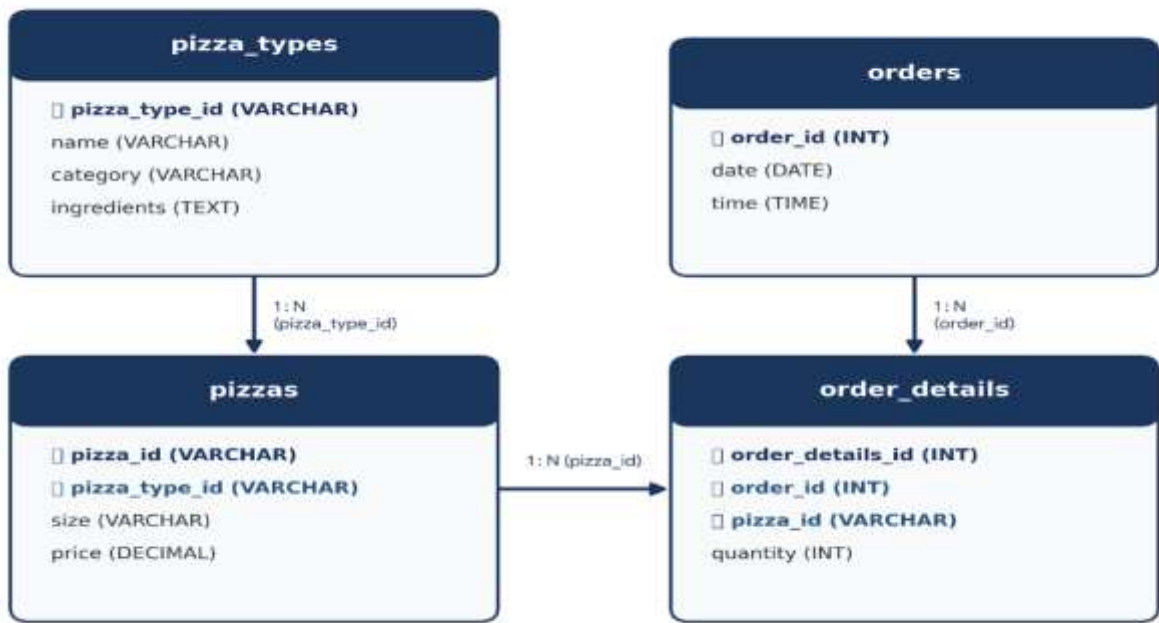
4. Functional Dependencies

The formal functional dependencies (FDs) justified strictly by our database schema design are summarized in the table below:

Table Name	Functional Dependency	Description / Justification
pizza_types	pizza_type_id → name, category, ingredients	The unique pizza type identifier determines the name, category, and list of ingredients.
pizzas	pizza_id → pizza_type_id, size, price	The unique pizza SKU code determines its specific size, price, and associated pizza category ID.
orders	order_id → date, time	The order ID uniquely determines the date and time the order was logged.
order_details	order_details_id → order_id, pizza_id, quantity	Each line item ID uniquely identifies the associated order, the ordered pizza SKU, and item quantity.

5. Table Relationships

The database schema implements clean relational linkages using Primary Key (PK) to Foreign Key (FK) references, as illustrated in the relationship diagram below:



Key Linkages Identified:

- 1. pizza_types to pizzas (1:N):** pizza_types.pizza_type_id (PK) → pizzas.pizza_type_id (FK). One pizza type can encompass multiple pizza size variants.
- 2. orders to order_details (1:N):** orders.order_id (PK) → order_details.order_id (FK). One customer order can contain multiple ordered line items.
- 3. pizzas to order_details (1:N):** pizzas.pizza_id (PK) → order_details.pizza_id (FK). A specific pizza SKU can appear across multiple order detail line items.

6. Normalization Work Done in Our Project

The provided Pizza Sales Database was already organized into separate relational tables in the uploaded project files. Therefore, instead of artificially claiming that the database was transformed through manual 1NF → 2NF → 3NF step-by-step conversion steps, our team performed an exhaustive analytical verification of the existing schema.

Specifically, our normalization work comprised:

- 1. Atomicity Audit:** Conducted an audit of every attribute to confirm field atomicity and absence of unnormalized repeating groups.
- 2. Dependency Mapping:** Identified all primary and foreign keys across all four project tables.
- 3. Normal Form Verification:** Formally mapped all functional dependencies to confirm the total absence of partial and transitive dependencies.
- 4. Integrity Check:** Verified that foreign key constraints properly preserve relational integrity across transactional tables.

7. Conclusion

Our normalization analysis leads to the following conclusions:

- **Normalization Status:** The Pizza Sales Database schema is fully normalized and complies with 1NF, 2NF, and 3NF standards.
- **Redundancy Elimination:** Because menu metadata (pizza_types and pizzas) is decoupled from transaction logs (orders and order_details), data duplication is completely minimized.
- **Data Integrity:** The structure prevents insertion, deletion, and update anomalies. Updating a pizza price or category name requires modifying exactly one record in the menu tables without impacting historical order logs.